

Day2 · 9차시

도구 사용 & MCP — 에이전트에 손발을 단다

말하는 에이전트에서 실제로 일하는 에이전트로.
7차시 ReAct 루프가 부르던 그 '행동(Act)'의 정체 = 도구(Tool).

SECTION 00

오프닝 — 왜 도구인가

LLM은 글은 잘 쓰는데, 계산·검색·외부 변경은 왜 못 믿나?

“LLM은 글을 잘 쓰는데 — 계산·검색·외부 시스템 변경은 왜 못 믿나?”

— 약점(산술·최신성·부작용)은 도구로 보완한다 = 오늘의 주제

지난 차시 → 오늘

루프는 배웠으니, 이제 손발을 단다

▶ 같은 맛집 MVP를 계속 확장 — 오늘 단 도구가 저녁 팀 프로젝트로 이어진다

7차시 · ReAct 루프

추론→행동→관찰
스스로 도는 에이전트

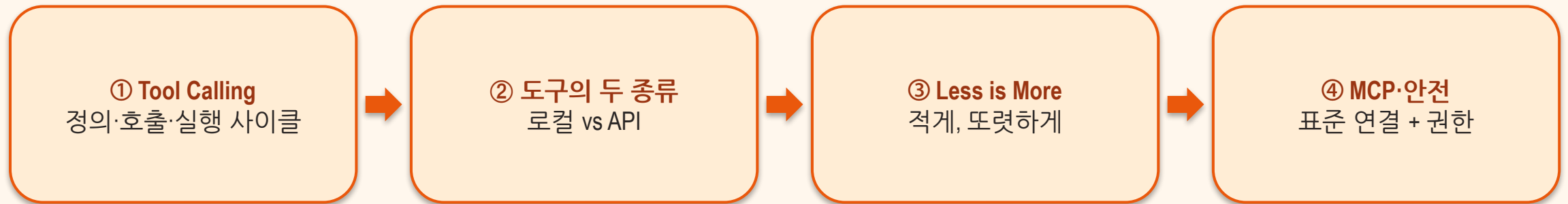
8차시 · 설계·UX

무엇을 맡길지
스코핑 · 신뢰의 설계

9차시(오늘) · 도구·MCP

그 '행동(Act)'의 정체
도구를 정의·연결한다

이 차시 동안 얻는 것



SECTION 01

Tool Calling — 도구를 부르는 한 사이클

도구(Tool)란 — 두 부류로 나뉜다

▶ 에이전트가 결과를 얻으려 실행하는 특정 능력·행동 — LLM(두뇌)이 정하면 도구(손발)가 한다

행동하는 도구 — Taking action

- order() — 주문 넣기
- set_meeting() — 일정 잡기
- run_code() — 코드 실행
- 세상을 **바꾼다** — 위험도 함께

VS

데이터를 얻는 도구 — Getting data

- weather() — 날씨 조회
- search() — 웹 검색
- wikipedia() — 지식 조회
- 세상을 **읽는다** — 상대적으로 안전

도구의 3층 구조 — 코드만이 도구가 아니다

▶ 모델은 코드를 못 본다 — 메타데이터가 로직만큼 중요한 이유

① 구현 코드

실제 로직 — 함수 본문 (모델에게는 보이지 않는다)

② 메타데이터

이름·설명·파라미터 스키마 — 모델이 도구를 고르는 유일한 단서

③ 호출 프로토콜

구조화된 요청/응답 규격 (JSON) — 모델과 코드가 주고받는 약속

도구는 '모듈'이다 — 함수 하나 = 도구 하나

▶ 도구를 바꿔도 루프(7차시)는 그대로 재사용

독립적

따로 개발·테스트·최적화
작게 나눌수록 다루기 쉽다

조합 가능

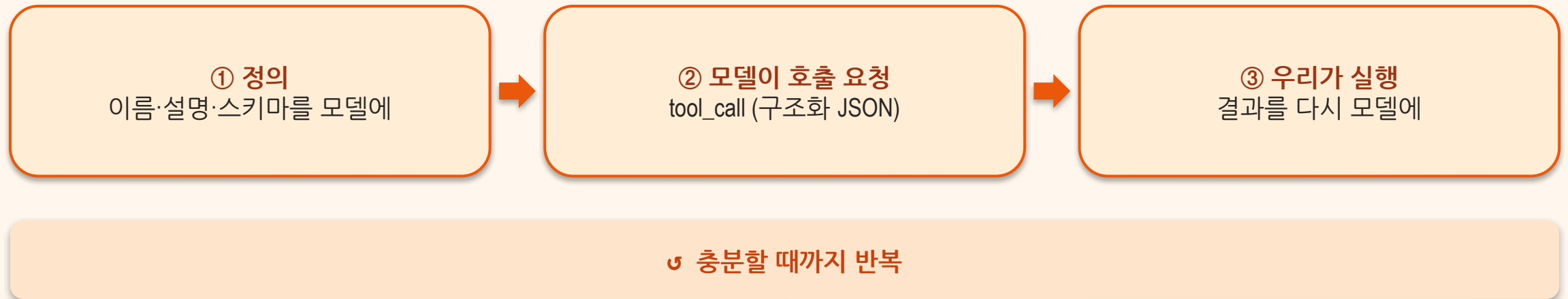
레고 블록처럼 조합해
복잡한 동작을 만든다

교체 가능

루프는 그대로 두고
도구만 갈아 끼운다

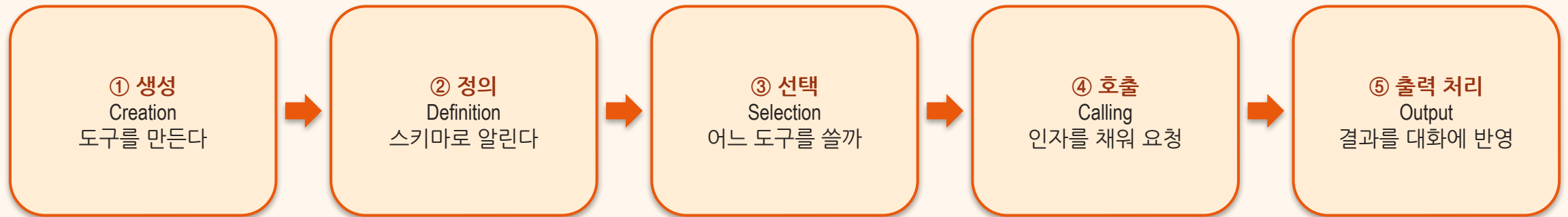
Tool Calling 3단계

▶ 모델은 '이 도구를, 이 인자로' 라는 구조화된 요청만 한다 — 실행은 우리 코드가



책의 5단계 — ‘선택’이 따로 있는 이유

▶ 가장 자주 틀리는 곳이 선택(Selection) — 그래서 별도 단계로 다룬다



정의 → 호출 → 실행

▶ 단순히 보여도 의미는 깊다 — 모델이 우리가 바인딩한 프로그램을 실행하게 된다

```
def add(x, y): # ① 도구(함수) 정의
    return x + y
TOOLS = [{"type": "function", "function": {
    "name": "add", "description": "두 정수를 더한다",
    "parameters": {"type": "object",
        "properties": {"x": {"type": "integer"}, "y": {"type": "integer"}}}}}]
# ② 모델 → add(x=11, y=49)   ③ 우리 → 60 을 messages에 다시
```

“임의의 프로그램을 모델 손에 쥐어준다” → 무엇을 쥐여줄지가 곧 책임 (안전 섹션에서 다시)

메타데이터가 로직만큼 중요하다

▶ 모델은 이름·설명·스키마만 보고 도구를 고른다 — 메타데이터 = 선택 정확도

나쁜 메타데이터 X

- 이름이 너무 일반적 — process
- 설명이 넓고 서로 겹침
- 스키마 느슨 — 타입·required 없음
- → 불필요할 때도 호출 · 오작동↑

VS

좋은 메타데이터 ✓

- 이름이 좁고 구체적 — calculate_sum
- 설명이 명확하고 서로 구별됨
- 스키마 엄격 — 타입·required 지정
- → 선택 정확도↑ · 오작동↓

SECTION 02

도구의 두 종류 — 로컬 vs API

로컬 도구 (Local)

▶ 결정적·수학적·오프라인 작업에 강하다

무엇

에이전트와 함께 배포되는, 사전 정의된 규칙·로직

보완 영역

모델이 약한 곳 — 산술·날짜/시간·단위 변환·정렬

강점

정밀성 · 예측가능성 · 단순성

대표 예

계산기 — LLM은 큰 수 곱셈을 틀려도 코드는 안 틀린다

API 기반 도구 (API-based)

▶ 모델 재학습 없이 기능 확장 + 최신 정보 접근

무엇

외부 서비스와 통신 → 실시간 데이터·로컬 불가능한 작업

대표 예

날씨·주가·번역·웹 검색·지도

강점

재학습 없이 기능이 늘어난다 + 최신성

주의

네트워크·인증·요금·실패 처리(타임아웃·재시도)

로컬 vs API — 결정 기준

▶ 둘은 배타적이지 않다 — 한 에이전트가 둘 다 가진다

로컬 도구

- 결정적·수학적 연산
- 오프라인·즉답
- 정밀·예측가능
- 예: 계산기·날짜·정렬

VS

API 도구

- 최신성·실시간 데이터
- 외부 시스템 연동
- 대용량·전문 연산
- 예: 검색·날씨·번역

독립적인 도구는 동시에 — 병렬 호출

▶ 가장 느린 도구만큼만 기다린다 — 순서 의존이 없으면 병렬로



SECTION 03

Less is More — 도구를 줄여 신뢰를 높인다

도구가 많을수록 헛갈린다

▶ ‘있으니까 다 붙인다’가 가장 흔한 함정

도구 12개 (영킴)

- 설명이 넓거나 겹침
- 오선택↑ · 지연↑
- ‘있으니까 다 붙인다’의 함정
- 확장성 나쁨 (building 5)

VS

핵심 3개 (또렷)

- 작업에 꼭 필요한 것만
- 경계가 또렷
- 선택 정확도↑ · 빠름
- 유지·디버깅 쉬움

적게, 또렷하게

▶ 8차시 스코핑의 도구 버전 — 권한·범위는 필요한 만큼만

적게

작업에 꼭 필요한 도구만 바인딩

또렷하게

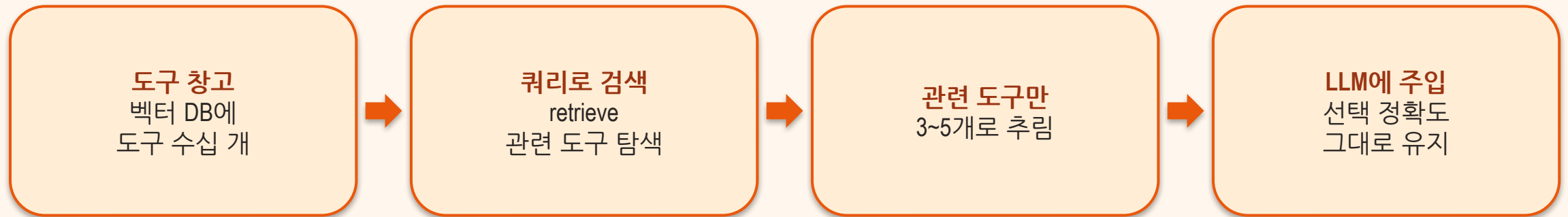
각 도구의 경계(범위)가 겹치지 않게

많이 필요하면

다 넣지 말고 → 검색으로 골라 주입 (다음 장)

도구가 수십 개라면 — 검색해서 골라 넣는다

▶ 전체 스키마를 다 넣지 않는다 — 쿼리와 관련된 도구만 주입 (RAG의 도구 버전)



SECTION 04

MCP — 표준으로 도구를 붙인다

MCP (Model Context Protocol)

▶ 비유: USB-C 같은 표준 포트 — 한 번 꽂으면 어디서나

무엇

LLM과 외부 도구·데이터를 잇는 **표준 약속** (JSON-RPC 2.0)

왜

도구마다 제각각 연결하던 것을 표준 하나로 → 재사용·확장 쉬움

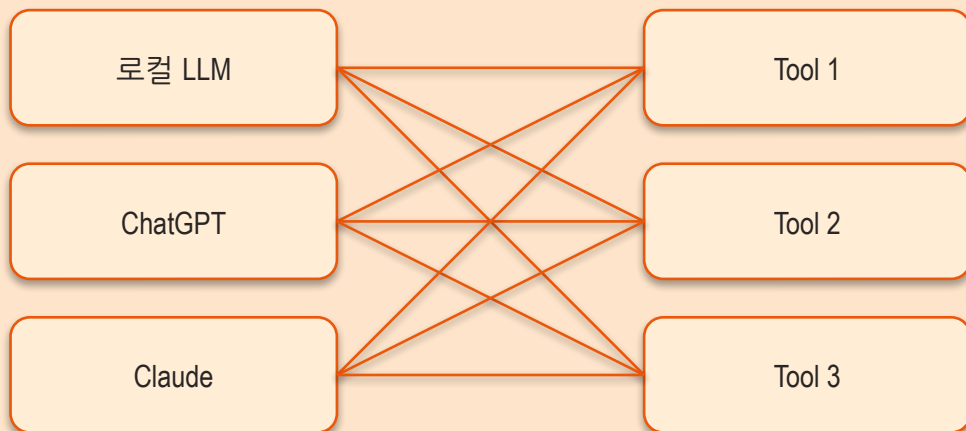
어디서 봤나

day1 2차시에서 등록 실습 — 오늘은 함수도구와 나란히 비교

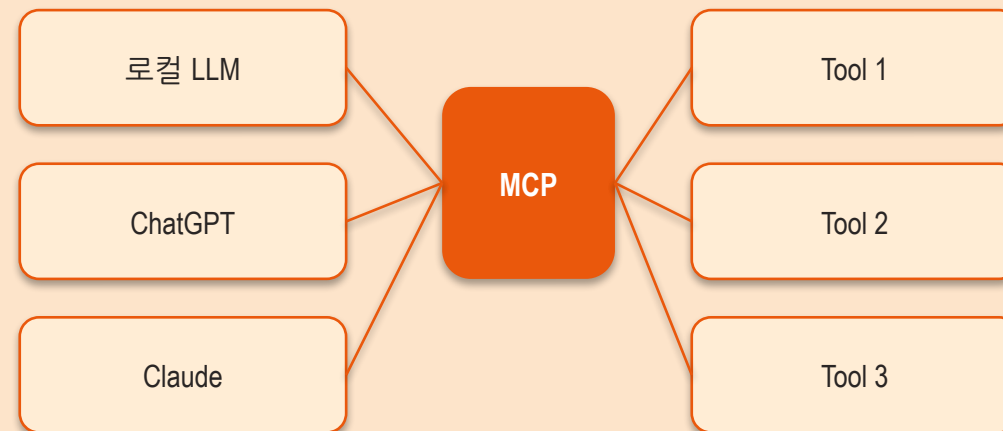
N×M 문제 → N+M 해법

▶ 모델 3개 × 도구 3개 = 맞춤 연결 9개 → MCP를 끼우면 3+3 = 6개

기존 — 도구마다 맞춤 연결



MCP — 표준 하나로



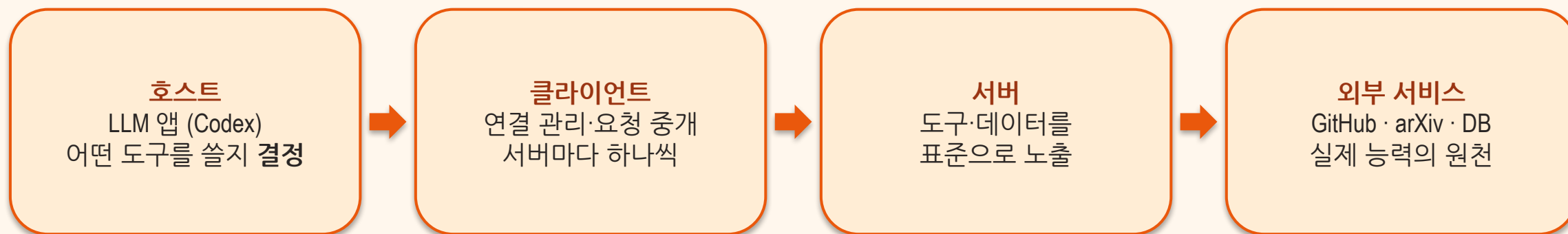
MCP 허브-스포크

▶ 중앙 에이전트가 표준(MCP) 하나로 여러 외부 서버에 연결 — 스포크는 갈아 끼운다



MCP 아키텍처 — 호스트 · 클라이언트 · 서버

▶ 각 층은 서로의 내부를 몰라도 된다 — 표준(JSON-RPC)으로만 대화



day1에서 배운 등록을 다시

▶ Codex CLI에 MCP 서버를 한 줄로 등록 — 그러면 에이전트가 그 도구를 쓸 수 있다

```
# 문서 검색 MCP(Context7) 등록
codex mcp add context7 -- npx -y @upstash/context7-mcp

# 브라우저 자동화 MCP(Playwright) 등록
codex mcp add playwright -- npx -y @playwright/mcp@latest

# 등록 확인
codex mcp list
```

MCP 3단계로 붙이기

▶ 함수도구(정의·호출·실행)와 같은 골격의 '표준판'

1

서버 등록

codex mcp add <이름> -- <실행명령>

2

권한 확인

/mcp 로 연결·도구 목록·권한 점검

3

호출

에이전트가 자연어 요청 중 필요할 때 자동 호출

함수 도구 vs MCP — 언제 무엇을

▶ 시작은 함수 도구로 — 공유·확장이 필요해지면 MCP로

함수 도구

- 내 앱 전용·간단·빠름
- 작은 능력(계산기 등)
- 코드 안에서 즉시 정의
- 처음 시작은 여기서

VS

MCP

- 표준화·재사용
- 여러 에이전트가 공유
- 외부 시스템·팀 공용
- 확장·재배포 부담 해결

SECTION 05

안전 — 권한 최소화 & 프롬프트 인젝션

도구를 바인딩한다 = 모델에 실행 권한을 준다

필요한 범위만 부여하라

▶ 도구 바인딩 = 모델에 실행 권한을 주는 일

① 읽기 (Read)

정보 조회 — 위험 낮음, 기본 허용

② 쓰기 (Write)

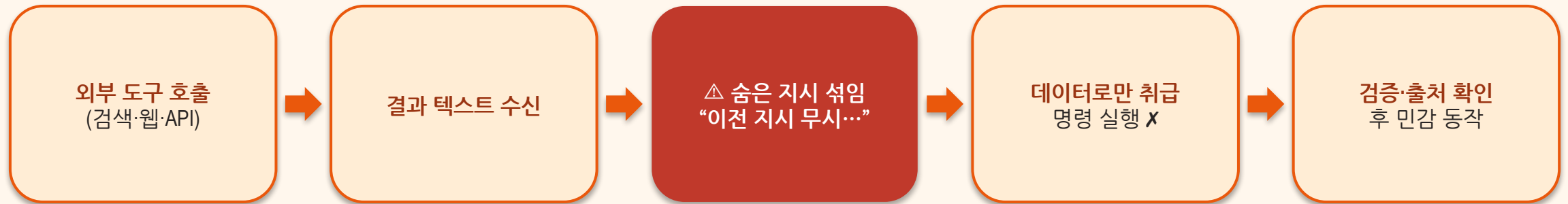
상태 변경 — 되돌릴 수 있는지 확인 후 신중히

③ 파괴적 (삭제·결제·전송)

사람 승인(Human-in-the-loop) 필수

도구 출력을 맹신하지 마라

▶ 도구 출력은 명령이 아니라 데이터 — 맹신 금지



상태를 바꾸는 도구, 3중으로 지킨다

▶ DB 삭제·API 변경처럼 세상을 바꾸는 도구는 사고도 실재가 된다 (building 4)

① 능력 최소화
도구 등록 시
좁은 범위 연산만 노출
(전체 DB 접근 ✕)

② 입력을 못 믿는다
입력 정화·파라미터 바인딩
최소 권한 DB 계정

③ 전부 기록한다
모든 도구 호출 로깅
이상 행동 실시간 경고

도구는 힘이다 — 줄수록 강해지고, 줄수록 위험해진다.

— 권한은 최소로, 출력은 데이터로, 파괴적 동작엔 사람 승인

SECTION 06

정리

오늘 3줄

- 도구 = 에이전트의 손발 — 정의→선택→호출→실행의 사이클, 메타데이터가 선택을 좌우
- 로컬(정밀)·API(실시간), 적게·또렷이 — 많아지면 검색해서 골라 주입
- MCP = **N+M 표준 연결** (호스트·클라이언트·서버), 권한은 최소·출력은 데이터로